

PC PART RETAIL AND CONFIGURATION STORE

Dependencies:

Back end: Java 25
Front end: JavaFX
Database: PostgreSQL
Connectivity: Neon

Classes:

FXML Controllers: -Build Controller
 -Cart Controller
 -Dashboard Controller
 -Login Controller
 -OrderHistoryController
 -SelectionController
 -UserBuildsController

Components: -Accessory
 -Build
 -Case
 -Cooler
 -CPU
 -GPU
 -Motherboard
 -PSU
 -Product
 -RAM
 -Storage
 -User
 -UserSession

DB: -Products: (includes a DAO for all classes listed above in the “Components” package)

 -DatabaseManager
 -ResultSetMapper

FXMLS: -build-dashboard.fxml
 -hello-view.fxml
 -dashboard.fxml
 -cart-view.fxml
 -selection-view.fxml
 -user-builds-view.fxml
 -my-orders-view.fxml

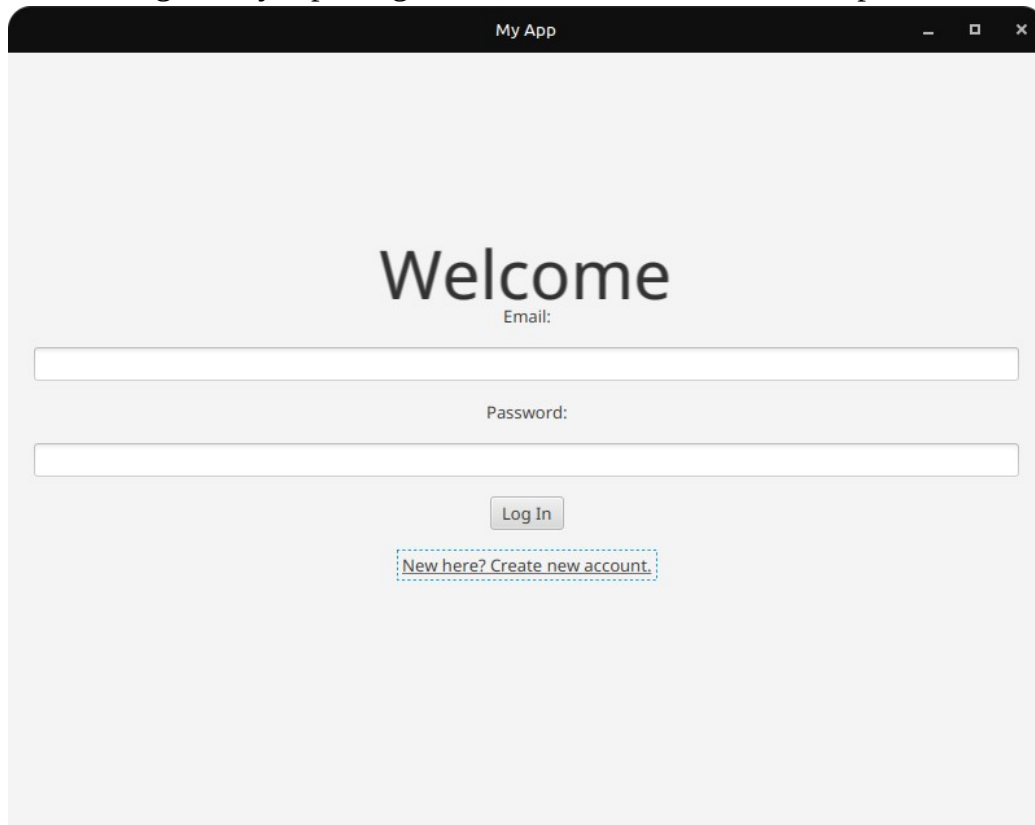
Models: -CartItem
 -Order

Full view of database diagram:

Behaviour:

This is a combination of a PC builder app and E-commerce platform where users can either shop for individual PC parts, build and order a custom PC, or both. This app is aimed at more intermediate to advanced PC builders. It checks for build errors, but will let you proceed further despite that with the intention that the user knows what they are doing.

To get started, the user is asked to log in with a preexisting email and password. If not, the user can register by inputting a username, email, address and password.



My App

Welcome

Email:

Password:

Log In

[New here? Create new account](#)

After the user is logged in, they are greeted to a dashboard consisting of various PC parts, with displayed images, product name and brand, category, price and current stock. The user also has the choice to view their current PC builds, the current session cart, their order history or to simply add an item to their cart. The images are displayed via the internet, but the app won't freeze to wait for images to load.

My App

Welcome, User

Search components...

Search

PC BUILDER

Browse Parts

My Builds

Cart

Orders

Logout

Image	Brand	Component Name	Category	Price	Stock	Add
	CoolerMaster	MasterBox V2	Case	99.99	9	+
	Sapphire	9060 XT	GPU	349.99	29	+
	HyperX	FURY 2x16 DDR5	RAM	219.99	18	+
	Thermaltake	Peerless Assassin 120	Cooler	69.99	45	+
	Gigabyte	AORUS ELITE B650M	Motherboard	174.99	40	+
	Corsair	TX 650	PSU	99.99	49	+
	ASUS	RTX 4090	GPU	799.99	2	+
	AMD	Ryzen 7 9800X3D	CPU	399.99	19	+
	Intel	I9 14900K	GPU	449.99	10	+
	Samsung	970 Evo 500GB	Storage	99.99	85	+
	SeaGate	Baracuda 3.5 inch 1TB	Storage	75.99	200	+
	Intel	Core i9-12900K	CPU	500.0	10	+

On the main page, the information is fetched from the “products” table to retrieve basic information about the parts themselves, via the ProductDAO class. Each DAO class uses a mapResultSet method from the that essentially looks for columns to map to a specific object’s attributes via Java’s Reflection property.

```

import ...

public class ResultSetMapper<T> { 27 usages  🧑 luca +1
    public List<T> mapResultSet(ResultSet rs, Class<T> clazz) { 9 usages  🧑 luca +1
        List<T> outputList = new ArrayList<>();
        try {
            while (rs.next()) {
                T dto = clazz.getDeclaredConstructor().newInstance();

                Class<?> currentClass = clazz;
                while (currentClass != null) {
                    Field[] fields = currentClass.getDeclaredFields();

                    for (Field field : fields) {
                        field.setAccessible(true);
                        String colName = toSnakeCase(field.getName());

                        try {
                            Object value = rs.getObject(colName);
                            if (value != null) {
                                field.set(dto, value);
                            }
                        } catch (Exception e){
                            System.out.println("Mapping Error for field: " + field.getName());
                            System.out.println("Looking for DB column: " + colName);
                            System.out.println("Error: " + e.getMessage());
                        }
                    }

                    currentClass = currentClass.getSuperclass();
                }
                outputList.add(dto);
            }
        } catch (Exception e) {
            System.out.println("Mapper Error: " + e.getMessage());
            e.printStackTrace();
        }
        return outputList;
    }

    private String toSnakeCase(String str) { return str.replaceAll( regex: "[a-z]([A-Z]+)", replacement: "$1_$2").toLowerCase(); }
}

```

COLUMNS [Add column](#)

```

id SERIAL PRIMARY KEY
manufacturer_id INTEGER
category_id INTEGER
name VARCHAR
price DOUBLE PRECISION
stock INTEGER
image_url TEXT

```

CONSTRAINTS [Add constraint](#)

```

CONSTRAINT products_manufacturers_fk FOREIGN KEY (manufacturer_id) REFERENCES public.manufacturers (id) ON UPDATE RESTRICT ON DELETE RESTRICT
CONSTRAINT products_product_categories_fk FOREIGN KEY (category_id) REFERENCES public.product_categories (id) ON UPDATE RESTRICT ON DELETE RESTRICT
CONSTRAINT products_pk PRIMARY KEY (id)

```

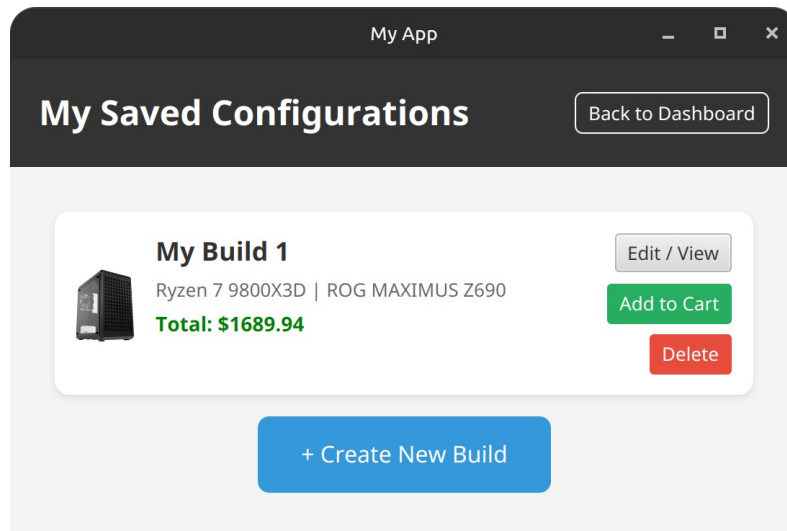
INDEXES [Add index](#)

```

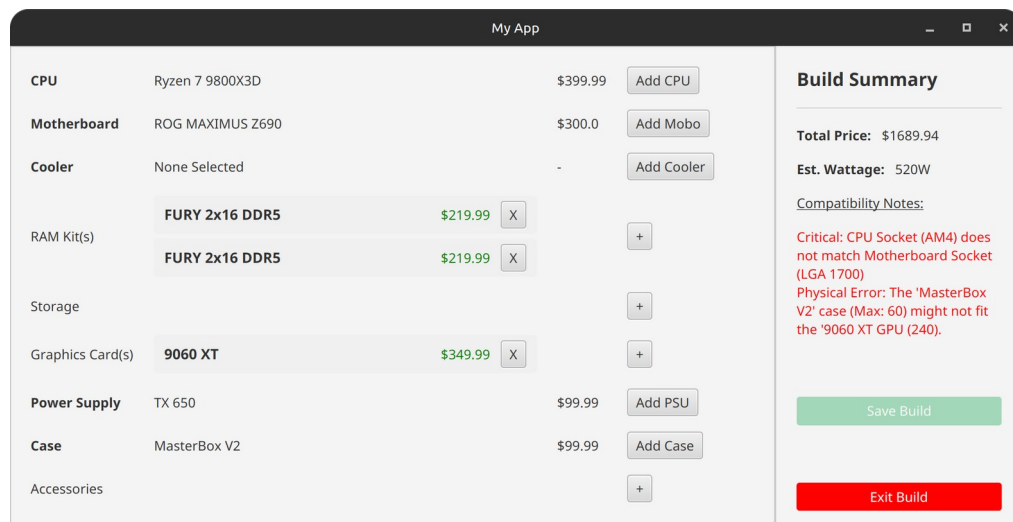
UNIQUE INDEX products_pk ... USING BTREE (id)

```

Moving further into the 2nd part of the app, we have the building interface. First, the user is introduced to their existing builds section where they can create a new build, edit, order or delete an existing one. The build card also displays the build name, the currently selected CPU and Motherboard and the total price of the entire build.

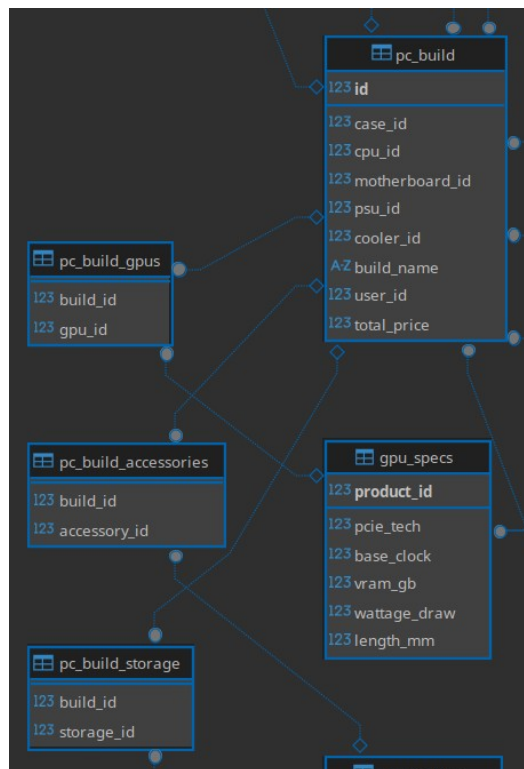


In the actual configurator interface let's the user select from the current database, one of each components, excluding the RAM, GPU, Storage and Accessories as these categories can be fitted as multiple quantities.



The app automatically checks general component compatibility, calculates estimated wattage and total price. However, the user can save and order the build regardless of any ongoing errors, because the app assumes the user knows what they are doing, or maybe current data could be outdated or wrong. Every component has a specific table where their specifications are listed.

To accommodate for multiple instances of components, I decided to split the `pc_build` table, holding standard components, into 5 different tables, the preexisting `pc_build` table, and the `pc_build_gpus/ram/accessories/storage` tables. After the user is done, they can click the “Save Build” button to save it into their builds storage.



After the user is done configuring their build or browsing through the current parts catalogue, they can choose to order whatever they want from the store. Once they are ready, they can hit the “Checkout” button, and the order will be sent to be processed.

